React Interview Coding Challenges

Section 1: Beginner Level Challenges (20 Challenges)

These challenges test fundamental React concepts including components, props, state, and basic hooks.

Challenge 1: Task Tracker Application

Create a task tracker application that allows users to manage their daily tasks with categories and priority levels.

Requirements:

1. **Task Management:**
   - Allow users to add new tasks with a title, description, category, and priority level (Low, Medium, High).
   - Display all tasks in a list format with visual indicators for priority (e.g., color-coded borders or badges).
   - Implement the ability to mark tasks as complete/incomplete with a checkbox.
   - Allow users to delete tasks with a confirmation prompt.
   - Show the total count of pending and completed tasks.
2. **Categories:**
   - Provide predefined categories: Work, Personal, Shopping, Health, and Other.
   - Allow filtering tasks by category using dropdown or tabs.
   - Display category name alongside each task.
3. **Priority Sorting:**

- Implement sorting functionality to arrange tasks by priority (High to Low or Low to High).
- Add a toggle button to switch between sort orders.

4. **Local Storage:**
   - Persist tasks in localStorage so they survive page refreshes.
   - Load saved tasks when the application mounts.
5. **UI/UX:**
   - Show an empty state message when no tasks exist.
   - Add smooth transitions when adding/removing tasks.
   - Make the interface responsive for mobile devices.

Challenge 2: Weather Dashboard

Build a weather dashboard that displays current weather information and a 5-day forecast for any city.

Requirements:

1. **Search Functionality:**
   - Provide a search input for users to enter a city name.
   - Show autocomplete suggestions as the user types (if API supports it).
   - Display an error message for invalid city names.
   - Remember the last searched city using localStorage.
2. **Current Weather Display:**
   - Show the city name, country, and current date/time.
   - Display current temperature with an option to toggle between Celsius and Fahrenheit.
   - Show weather condition (sunny, cloudy, rainy, etc.) with appropriate icons.

- Display additional details: humidity, wind speed, and "feels like" temperature.

3. **5-Day Forecast:**
   - Display a 5-day forecast below the current weather.
   - Each day should show: day name, weather icon, high/low temperatures.
   - Allow users to click on a forecast day to see hourly breakdown.
4. **Recent Searches:**
   - Maintain a list of the last 5 searched cities.
   - Allow users to quickly select from recent searches.
   - Provide option to clear search history.
5. **API Integration:**
   - Use OpenWeatherMap API or similar weather API.
   - Handle loading states with skeleton loaders.
   - Implement proper error handling for network failures.

Challenge 3: Interactive Quiz Application

Create an interactive quiz application with multiple categories, timer, and score tracking.

Requirements:

1. **Quiz Setup:**
   - Allow users to select a quiz category (General Knowledge, Science, History, Sports, Entertainment).
   - Let users choose difficulty level (Easy, Medium, Hard).
   - Provide option to select number of questions (5, 10, 15, or 20).
   - Display a "Start Quiz" button that begins the quiz.
2. **Question Display:**

- Show one question at a time with multiple choice answers (4 options).
- Display current question number and total questions (e.g., "Question 3 of 10").
- Implement a countdown timer for each question (30 seconds default).
- Highlight selected answer before confirming.

3. **Answer Handling:**
   - Show immediate feedback when an answer is selected (correct/incorrect).
   - If wrong, highlight the correct answer in green.
   - Disable answer selection after choosing.
   - Auto-advance to next question after 2 seconds or allow manual navigation.
4. **Score & Results:**
   - Track and display current score throughout the quiz.
   - Show final results page with: total score, percentage, time taken.
   - Display a breakdown of correct/incorrect answers.
   - Provide option to review all questions with correct answers.
   - Allow users to restart quiz or choose new category.
5. **API Integration:**
   - Use Open Trivia Database API (opentdb.com) for questions.
   - Handle API errors gracefully.

Challenge 4: Expense Tracker with Charts

Build an expense tracker that helps users monitor their spending with visual charts and budget management.

Requirements:

1. **Expense Entry:**
   - Allow users to add expenses with: amount, description, category, and date.

- Categories should include: Food, Transportation, Entertainment, Shopping, Bills, Healthcare, Other.
- Validate that amount is a positive number.
- Allow editing and deleting existing expenses.

2. **Expense Display:**
   - Show a list of all expenses sorted by date (newest first).
   - Display category icons/colors for visual distinction.
   - Implement filtering by date range (This Week, This Month, Custom Range).
   - Add search functionality to find expenses by description.
3. **Budget Management:**
   - Allow users to set a monthly budget.
   - Show remaining budget with a progress bar.
   - Display warning when spending exceeds 80% of budget.
   - Show alert when budget is exceeded.
4. **Charts & Analytics:**
   - Display a pie chart showing expense distribution by category.
   - Show a bar chart comparing daily/weekly spending.
   - Display total expenses for the selected period.
   - Calculate and show average daily spending.
5. **Data Persistence:**
   - Store all expenses in localStorage.
   - Implement export functionality to download expenses as CSV.

Challenge 5: Recipe Finder Application

Create a recipe finder application where users can search for recipes, save favorites, and create shopping lists.

Requirements:

1. **Recipe Search:**
    - Provide a search bar to find recipes by name or ingredient.
    - Display search results in a grid layout with recipe images.
    - Show recipe name, cooking time, and difficulty level on cards.
    - Implement pagination or infinite scroll for large result sets.
2. **Recipe Details:**
    - Create a detailed view for each recipe showing:
        - Full-size image
        - Complete ingredient list with quantities
        - Step-by-step cooking instructions
        - Nutritional information (if available)
        - Serving size with option to adjust quantities
3. **Favorites System:**
    - Allow users to save recipes to favorites.
    - Display a dedicated favorites page.
    - Persist favorites in localStorage.
    - Allow removing recipes from favorites.
4. **Shopping List:**
    - Let users add recipe ingredients to a shopping list.
    - Allow manual addition of items to the list.
    - Implement checkbox to mark items as purchased.
    - Group items by category (Produce, Dairy, Meat, etc.).

5. **API Integration:**
    - Use Spoonacular API or TheMealDB API.
    - Handle loading and error states appropriately.

Challenge 6: Pomodoro Timer with Task Integration

Build a Pomodoro timer application with task management and productivity statistics.

Requirements:

1. **Timer Functionality:**
    - Implement a 25-minute work timer (Pomodoro).
    - Add 5-minute short break and 15-minute long break timers.
    - Display time remaining in MM:SS format with circular progress indicator.
    - Provide Start, Pause, Resume, and Reset controls.
    - Play audio notification when timer completes.
2. **Task Integration:**
    - Allow users to create tasks to work on during Pomodoros.
    - Link current Pomodoro session to a specific task.
    - Track how many Pomodoros each task has consumed.
    - Mark tasks as complete when finished.
3. **Session Management:**
    - Follow standard Pomodoro technique: 4 work sessions, then long break.
    - Show current session number (e.g., "Session 2 of 4").
    - Auto-start break timer after work session (optional setting).
    - Allow skipping breaks.
4. **Statistics:**

- Track daily Pomodoro count.
- Show weekly summary of completed Pomodoros.
- Display total focus time for the day/week.
- Store statistics in localStorage.

5. **Customization:**
   - Allow users to customize timer durations.
   - Provide different notification sounds.
   - Add dark/light mode toggle.

Challenge 7: Image Gallery with Lightbox

Create a responsive image gallery with lightbox functionality, filtering, and lazy loading.

Requirements:

1. **Gallery Display:**
   - Display images in a responsive masonry or grid layout.
   - Show image thumbnails with hover effects.
   - Display image title/caption on hover.
   - Implement lazy loading for images.
2. **Lightbox:**
   - Open full-size image in modal when clicked.
   - Support keyboard navigation (arrow keys, escape).
   - Add previous/next buttons for navigation.
   - Show image counter (e.g., "3 of 15").
   - Support swipe gestures on mobile.
3. **Categories & Filtering:**

- Organize images into categories (Nature, Architecture, People, etc.).
- Allow filtering by category with animated transitions.
- Show "All" option to display all images.

4. **Search:**
   - Implement search by image title or tags.
   - Highlight matching images.
   - Show "No results" message when appropriate.
5. **Upload (Mock):**
   - Create upload form with drag-and-drop support.
   - Show upload progress indicator.
   - Allow adding title, description, and category to uploaded images.

Challenge 8: Notes Application with Markdown

Build a notes application that supports Markdown editing with live preview.

Requirements:

1. **Note Management:**
   - Create, edit, and delete notes.
   - Display notes in a sidebar list.
   - Show note title and preview text in list.
   - Sort notes by last modified date.
   - Search notes by title or content.
2. **Markdown Editor:**
   - Implement split view: editor on left, preview on right.
   - Support common Markdown: headings, bold, italic, lists, links, images, code blocks.

- Add toolbar buttons for common formatting.
- Implement syntax highlighting in editor.

3. **Organization:**
   - Allow creating folders/notebooks to organize notes.
   - Implement drag-and-drop to move notes between folders.
   - Add tags to notes for additional organization.
   - Filter notes by folder or tag.
4. **Data Persistence:**
   - Save notes to localStorage.
   - Auto-save while typing (debounced).
   - Show last saved timestamp.
5. **Export:**
   - Export notes as Markdown files.
   - Export notes as HTML.
   - Allow copying note content to clipboard.

Challenge 9: Countdown Timer App

Create a customizable countdown timer for events with multiple timers support.

Requirements:

1. **Timer Creation:**
   - Allow creating named timers with target date/time.
   - Support countdown to specific events (birthdays, holidays, deadlines).
   - Validate that target date is in the future.
   - Allow setting recurring timers (weekly, monthly, yearly).

2. **Timer Display:**
    - Show days, hours, minutes, seconds remaining.
    - Use animated flip-card or digital display style.
    - Display timer name and target date.
    - Show progress bar or percentage complete.
3. **Multiple Timers:**
    - Support multiple active timers simultaneously.
    - Display timers in a grid or list view.
    - Allow reordering timers by drag-and-drop.
    - Pin important timers to top.
4. **Notifications:**
    - Play sound when timer reaches zero.
    - Show browser notification (with permission).
    - Allow snooze option for recurring reminders.
5. **Styling:**
    - Provide multiple theme options.
    - Allow custom colors for each timer.
    - Responsive design for all screen sizes.

Challenge 10: Contact Manager

Build a contact manager application with CRUD operations and contact grouping.

Requirements:

1. **Contact Information:**
    - Store: name, email, phone, address, company, notes.

- Support multiple phone numbers and emails per contact.
- Allow adding profile picture (URL or file upload mock).
- Validate email and phone formats.

2. **Contact Display:**
   - Show contacts in list or card view (toggleable).
   - Display avatar with initials if no image.
   - Sort contacts alphabetically by name.
   - Show quick actions on hover (call, email, edit, delete).
3. **Groups:**
   - Create contact groups (Family, Friends, Work, etc.).
   - Assign contacts to multiple groups.
   - Filter contacts by group.
   - Show group count badges.
4. **Search & Filter:**
   - Search contacts by name, email, or phone.
   - Filter by group or favorites.
   - Implement alphabet quick-jump navigation.
5. **Import/Export:**
   - Export contacts as JSON or CSV.
   - Import contacts from JSON file.
   - Merge duplicate contacts.

Challenge 11: Movie Search Application

Create a movie search application using TMDB or OMDB API.

Requirements:

1. **Search:**
   - Search movies by title.
   - Show search results with poster, title, year, rating.
   - Implement debounced search (avoid excessive API calls).
   - Display "No results found" appropriately.
2. **Movie Details:**
   - Show detailed view with: poster, backdrop, plot, cast, runtime, genres.
   - Display ratings from multiple sources.
   - Show similar movie recommendations.
   - Link to trailer (YouTube embed or link).
3. **Watchlist:**
   - Add/remove movies from watchlist.
   - Display watchlist in separate tab.
   - Persist watchlist in localStorage.
   - Show watched/unwatched status.
4. **Filtering:**
   - Filter by genre, year, rating.
   - Sort by popularity, rating, release date.
   - Show trending movies on homepage.
5. **Responsive Design:**
   - Grid layout for results.
   - Mobile-friendly detail view.
   - Swipeable movie cards on mobile.

Challenge 12: Habit Tracker

Build a habit tracking application to help users build and maintain daily habits.

Requirements:

1. **Habit Management:**
   - Create habits with name, description, frequency (daily/weekly).
   - Set target (e.g., "3 times per week").
   - Assign color and icon to each habit.
   - Edit and delete habits.
2. **Tracking:**
   - Mark habits as complete for each day.
   - Display calendar view showing completion status.
   - Show current streak for each habit.
   - Calculate and display completion percentage.
3. **Dashboard:**
   - Show today's habits with completion status.
   - Display overall progress for the week/month.
   - Highlight longest streak achievements.
   - Show motivational messages based on progress.
4. **Statistics:**
   - Chart showing weekly/monthly completion rates.
   - Best performing habits.
   - Areas needing improvement.
   - Export statistics as image or PDF.
5. **Reminders:**
   - Set reminder time for each habit.
   - Browser notification support.

- Snooze or dismiss reminders.

Challenge 13: Currency Converter

Build a currency converter with real-time exchange rates.

Requirements:

1. **Conversion:**
   - Input amount to convert.
   - Select source and target currencies.
   - Display converted amount instantly.
   - Support swap button to switch currencies.
2. **Currency Support:**
   - Support at least 30 major currencies.
   - Show currency flag and symbol.
   - Display full currency name.
   - Search/filter currency list.
3. **Historical Rates:**
   - Show rate chart for selected pair (30-day history).
   - Display rate change percentage.
   - Mark high/low points on chart.
4. **Favorites:**
   - Save favorite currency pairs.
   - Quick access to frequently used conversions.
   - Show all favorites on dashboard.
5. **API Integration:**

- Use Exchange Rate API or similar.
- Handle API rate limits.
- Cache rates to reduce API calls.
- Show last updated timestamp.

Challenge 14: Blog Post Creator

Create a simple blog post creation and viewing application.

Requirements:

1. **Post Creation:**
   - Create posts with title, content, category, tags.
   - Rich text editor for content.
   - Support image embedding.
   - Save as draft or publish immediately.
2. **Post Listing:**
   - Display posts in blog-style layout.
   - Show featured image, title, excerpt, date, author.
   - Filter by category or tag.
   - Pagination or infinite scroll.
3. **Post Detail:**
   - Full post view with formatted content.
   - Display author info and publish date.
   - Show related posts.
   - Estimated reading time.
4. **Comments (Mock):**

- Add comments to posts.
- Display comment count.
- Reply to comments.
- Like/upvote comments.

5. **Search:**
   - Search posts by title or content.
   - Highlight search terms in results.
   - Filter search by date range.

Challenge 15: Flashcard Study App

Build a flashcard application for studying and memorization.

Requirements:

1. **Card Management:**
   - Create flashcards with front (question) and back (answer).
   - Support text, images, and code snippets.
   - Organize cards into decks/topics.
   - Import/export decks as JSON.
2. **Study Mode:**
   - Show card front, click/tap to flip and reveal answer.
   - Mark cards as "Know" or "Don't Know".
   - Shuffle card order.
   - Track study progress per deck.
3. **Spaced Repetition:**
   - Implement simple spaced repetition algorithm.

- Show cards due for review.
- Display next review date.
- Prioritize difficult cards.

4. **Progress Tracking:**
   - Show mastery level per card/deck.
   - Display study streak.
   - Chart progress over time.
   - Set daily study goals.
5. **Sharing:**
   - Share deck link (mock).
   - Browse public decks.
   - Clone decks to personal collection.

Challenge 16: Bookmark Manager

Create a bookmark manager with folders, tags, and search.

Requirements:

1. **Bookmark Creation:**
   - Add bookmarks with URL, title, description.
   - Auto-fetch page title and favicon.
   - Assign to folder and add tags.
   - Support bulk import from browser export.
2. **Organization:**
   - Create nested folder structure.
   - Drag-and-drop to reorganize.

- Tag-based organization.
- Mark favorites for quick access.

3. **Display:**
   - List or grid view toggle.
   - Show favicon, title, URL preview.
   - Display last visited date.
   - Show broken link indicators.
4. **Search & Filter:**
   - Search by title, URL, or tags.
   - Filter by folder or tag.
   - Sort by date added, last visited, alphabetical.
   - Recent bookmarks section.
5. **Export:**
   - Export as HTML (browser compatible).
   - Export as JSON.
   - Selective export by folder.

Challenge 17: Password Generator

Build a secure password generator with customization options.

Requirements:

1. **Generation Options:**
   - Set password length (8-128 characters).
   - Include/exclude: uppercase, lowercase, numbers, symbols.
   - Exclude ambiguous characters option (0, O, l, 1).

- Custom character exclusion list.

2. **Password Display:**
   - Show generated password clearly.
   - Copy to clipboard button.
   - Regenerate button.
   - Password strength indicator.
3. **Password History:**
   - Store last 10 generated passwords (optional, with warning).
   - Copy from history.
   - Clear history button.
4. **Strength Meter:**
   - Visual strength indicator (weak, medium, strong).
   - Estimate crack time.
   - Suggestions for improvement.
5. **Presets:**
   - Save favorite configurations as presets.
   - Quick-select common patterns.
   - PIN generator mode (numbers only).

Challenge 18: Unit Converter

Create a comprehensive unit converter for various measurement types.

Requirements:

1. **Conversion Categories:**
   - Length (km, m, cm, mm, miles, feet, inches).

- Weight (kg, g, mg, pounds, ounces).
- Temperature (Celsius, Fahrenheit, Kelvin).
- Volume (liters, ml, gallons, cups).
- Time (seconds, minutes, hours, days).
- Data (bytes, KB, MB, GB, TB).

2. **Conversion Interface:**
   - Input value with unit selection.
   - Show conversion to all units in category.
   - Highlight the target conversion.
   - Support decimal precision settings.
3. **Favorites:**
   - Save frequently used conversions.
   - Quick access on homepage.
   - Recent conversions history.
4. **Formula Display:**
   - Show conversion formula used.
   - Educational explanations.
   - Reference tables.
5. **UI/UX:**
   - Category tabs or dropdown.
   - Instant conversion as you type.
   - Swap units button.
   - Mobile-optimized number pad.

Challenge 19: Event Countdown Cards

Build an application to create and display beautiful event countdown cards.

Requirements:

1. **Card Creation:**
   - Create cards with event name, date, description.
   - Choose from template designs.
   - Upload background image or select from presets.
   - Customize colors and fonts.
2. **Countdown Display:**
   - Show days, hours, minutes, seconds.
   - Animate number changes.
   - Display event details on card.
   - Show "Event Started!" when complete.
3. **Card Gallery:**
   - Display all cards in gallery view.
   - Filter by upcoming/past events.
   - Sort by date.
   - Archive completed events.
4. **Sharing:**
   - Download card as image.
   - Copy shareable link (mock).
   - Share to social media (mock).
5. **Categories:**
   - Birthdays, Holidays, Deadlines, Custom.
   - Category-specific default themes.
   - Filter by category.

Challenge 20: Testimonial Slider

Create a testimonial carousel/slider component with various animations.

Requirements:

1. **Testimonial Display:**
   - Show testimonial text, author name, company, photo.
   - Support long and short testimonials.
   - Display rating stars if applicable.
   - Quote styling with decorative elements.
2. **Navigation:**
   - Previous/Next buttons.
   - Dot indicators for direct navigation.
   - Auto-play with pause on hover.
   - Swipe support on mobile.
3. **Animations:**
   - Multiple transition options (slide, fade, flip).
   - Smooth animation between testimonials.
   - Configurable animation speed.
4. **Configuration:**
   - Set number of visible testimonials.
   - Auto-play interval setting.
   - Loop or stop at end option.
   - Responsive breakpoint settings.
5. **Admin Panel:**
   - Add/edit/delete testimonials.

- Reorder testimonials.
- Preview changes live.

Section 2: Mid-Level Challenges (22 Challenges)

These challenges require intermediate React skills including Context API, custom hooks, performance optimization, and API integration.

Challenge 21: E-Commerce Product Catalog

Build a product catalog with filtering, sorting, and cart functionality using React Context for state management.

Requirements:

1. **Product Listing:**
   - Display products in a grid with images, names, prices, and ratings.
   - Implement infinite scroll or pagination.
   - Show product quick-view on hover.
   - Display "Sale" or "New" badges where applicable.
2. **Filtering & Sorting:**
   - Filter by: category, price range, rating, availability.
   - Multiple active filters simultaneously.
   - Sort by: price (low/high), rating, newest, popularity.

- Show active filters with remove option.
- Display result count.

3. **Shopping Cart (Context API):**
   - Add products to cart with quantity selection.
   - Update quantities in cart.
   - Remove items from cart.
   - Show cart item count in header.
   - Display cart total with tax calculation.
   - Persist cart in localStorage.
4. **Product Detail:**
   - Full product page with multiple images.
   - Image zoom on hover.
   - Size/color variant selection.
   - Related products section.
   - Customer reviews display.
5. **Wishlist:**
   - Add/remove from wishlist.
   - Move items from wishlist to cart.
   - Share wishlist (mock).

Challenge 22: Kanban Board Application

Create a Kanban-style project management board with drag-and-drop functionality.

Requirements:

1. **Board Structure:**

- Multiple columns (To Do, In Progress, Review, Done).
- Add, rename, and delete columns.
- Reorder columns via drag-and-drop.
- Column task count display.

2. **Task Cards:**

- Create tasks with title, description, due date, priority, assignee.
- Support labels/tags with colors.
- Show checklist progress on card.
- Display due date with overdue highlighting.
- Task card quick edit.

3. **Drag and Drop:**

- Drag tasks between columns.
- Drag to reorder within column.
- Visual feedback during drag.
- Smooth animations.

4. **Task Details Modal:**

- Full task editing in modal.
- Checklist management.
- Activity log showing changes.
- Comments on tasks.
- Attachments (mock upload).

5. **Filters & Search:**

- Filter by assignee, label, or due date.
- Search tasks by title or description.
- Show/hide completed tasks.

Challenge 23: Social Media Feed

Build a social media feed with infinite scroll, likes, comments, and real-time updates simulation.

Requirements:

1. **Feed Display:**
   - Show posts with author avatar, name, timestamp.
   - Support text, images, and videos in posts.
   - Display like count and comment count.
   - Infinite scroll loading.
   - Show relative timestamps ("2 hours ago").
2. **Interactions:**
   - Like/unlike posts with animation.
   - Comment on posts.
   - Share posts (mock).
   - Save/bookmark posts.
   - Report post option.
3. **Post Creation:**
   - Create new post form.
   - Support text with character limit.
   - Add images/videos (mock upload).
   - Tag people (autocomplete).
   - Add location.
4. **Comments System:**
   - Nested replies to comments.
   - Like comments.
   - Load more comments button.

- Optimistic updates for instant feedback.

5. **Real-time Simulation:**
   - Show "New posts available" notification.
   - Update like counts in real-time (polling).
   - Typing indicator in comments.

Challenge 24: Real-Time Chat Interface

Create a chat application interface with conversations, messages, and online status.

Requirements:

1. **Conversation List:**
   - Show all conversations with last message preview.
   - Display unread message count.
   - Online/offline status indicators.
   - Search conversations by name.
   - Pin important conversations.
2. **Message Display:**
   - Show messages in chat bubble format.
   - Display sent/delivered/read status.
   - Support text, images, files.
   - Group messages by date.
   - Show typing indicator.
3. **Message Input:**
   - Text input with emoji picker.
   - File attachment button.

- Voice message button (mock).
- Send on Enter, new line on Shift+Enter.

4. **User Profile:**
   - View contact profile in sidebar.
   - Show shared media.
   - Mute/block options.
   - Clear conversation history.
5. **Notifications:**
   - Desktop notification for new messages.
   - Sound for incoming messages.
   - Do not disturb mode.

Challenge 25: Music Player Application

Build a music player with playlist management and audio controls.

Requirements:

1. **Player Controls:**
   - Play, pause, next, previous buttons.
   - Progress bar with seek functionality.
   - Volume control with mute toggle.
   - Shuffle and repeat modes.
   - Current song info display.
2. **Playlist Management:**
   - Create and delete playlists.
   - Add/remove songs from playlists.

- Reorder songs via drag-and-drop.
- Save playlists to localStorage.

3. **Now Playing:**
   - Display album art.
   - Show song title, artist, album.
   - Lyrics display (mock data).
   - Queue management.
4. **Library:**
   - Browse by songs, albums, artists.
   - Search across library.
   - Sort by name, date added, play count.
   - Recently played section.
5. **Mini Player:**
   - Collapsible mini player mode.
   - Keyboard shortcuts for controls.
   - Media session integration (browser).

Challenge 26: Appointment Booking System

Create an appointment booking system with calendar view and time slot selection.

Requirements:

1. **Calendar View:**
   - Monthly calendar with available dates highlighted.
   - Show existing appointments on calendar.
   - Navigate between months.

- Disable past dates.

2. **Time Slots:**
   - Display available time slots for selected date.
   - Show slot duration and price.
   - Block already booked slots.
   - Support multiple service providers.
3. **Booking Form:**
   - Select service type.
   - Choose provider (if applicable).
   - Enter customer details.
   - Add notes for appointment.
   - Confirmation screen.
4. **My Appointments:**
   - List upcoming appointments.
   - Show appointment details.
   - Cancel/reschedule option.
   - Add to calendar download.
5. **Admin View:**
   - Manage availability schedule.
   - View all bookings.
   - Confirm/cancel appointments.
   - Set blocked dates (holidays).

Challenge 27: File Manager Interface

Build a file manager with folder navigation, file operations, and preview.

Requirements:

1. **File Display:**
   - Grid and list view toggle.
   - Show file icons by type.
   - Display file name, size, modified date.
   - Thumbnail previews for images.
   - Sort by name, size, date, type.
2. **Navigation:**
   - Folder tree in sidebar.
   - Breadcrumb navigation.
   - Back/forward navigation history.
   - Quick access to recent files.
3. **File Operations:**
   - Create new folder.
   - Rename files/folders.
   - Delete with confirmation.
   - Move via drag-and-drop.
   - Copy/paste functionality.
   - Multi-select for bulk operations.
4. **Preview:**
   - Image preview in modal.
   - Text file content preview.
   - PDF preview (mock).
   - Audio/video preview.
5. **Search:**
   - Search files by name.
   - Filter by file type.

- Search within current folder or all.

Challenge 28: Survey Builder

Create a survey/form builder with drag-and-drop question creation.

Requirements:

1. **Question Types:**
   - Multiple choice (single/multi select).
   - Text input (short/long answer).
   - Rating scale.
   - Date picker.
   - File upload field.
   - Dropdown selection.
2. **Survey Builder:**
   - Drag-and-drop to add/reorder questions.
   - Edit question text and options.
   - Mark questions as required.
   - Add descriptions/help text.
   - Preview survey in real-time.
3. **Survey Taking:**
   - Clean form interface.
   - Progress indicator.
   - Validation messages.
   - Navigate between questions.
   - Submit confirmation.
4. **Responses:**

- View all responses.
- Response summary charts.
- Export to CSV.
- Individual response view.

5. **Settings:**
   - Survey title and description.
   - Start/end date.
   - Response limit.
   - Thank you message customization.

Challenge 29: Dashboard with Widgets

Build a customizable dashboard with draggable and resizable widgets.

Requirements:

1. **Widget Types:**
   - Statistics cards.
   - Line/bar/pie charts.
   - Recent activity list.
   - Todo widget.
   - Calendar widget.
   - Weather widget.
2. **Layout Customization:**
   - Drag widgets to rearrange.
   - Resize widgets.
   - Add/remove widgets.
   - Save layout to localStorage.

- Reset to default layout.

3. **Widget Configuration:**
   - Configure data source for each widget.
   - Set refresh interval.
   - Choose chart colors.
   - Set date range for data.
4. **Responsive:**
   - Different layouts for different screen sizes.
   - Collapse widgets to cards on mobile.
   - Maintain usability on all devices.
5. **Themes:**
   - Light and dark theme.
   - Custom accent colors.
   - Widget-specific styling options.

Challenge 30: Multi-Step Form Wizard

Create a multi-step form with validation, progress tracking, and review.

Requirements:

1. **Steps:**
   - Personal Information (name, email, phone).
   - Address Details (street, city, state, zip, country).
   - Preferences (checkboxes, radio buttons, dropdowns).
   - Review & Submit.
2. **Navigation:**

- Next/Previous buttons.
- Step indicator showing progress.
- Click on completed steps to navigate.
- Disable future steps until current is valid.

3. **Validation:**
   - Validate each step before proceeding.
   - Show inline error messages.
   - Highlight invalid fields.
   - Custom validation rules per field.
4. **Review Step:**
   - Summary of all entered data.
   - Edit buttons to jump to specific steps.
   - Terms and conditions checkbox.
   - Submit button.
5. **State Persistence:**
   - Save progress to localStorage.
   - Resume from where left off.
   - Clear saved data on submit.

Challenge 31: Recipe Meal Planner

Build a weekly meal planner with recipe integration and shopping list generation.

Requirements:

1. **Weekly Calendar:**
   - 7-day view with breakfast, lunch, dinner slots.

- Drag recipes to meal slots.
- Copy meals between days.
- Clear day/week option.

2. **Recipe Integration:**
   - Browse recipes from library.
   - Search and filter recipes.
   - Add custom recipes.
   - Show nutritional summary per day.
3. **Shopping List:**
   - Auto-generate from meal plan.
   - Combine duplicate ingredients.
   - Group by category.
   - Check off purchased items.
   - Add manual items.
4. **Meal Templates:**
   - Save weekly plans as templates.
   - Load from templates.
   - Share templates (mock).
5. **Nutrition Tracking:**
   - Daily calorie total.
   - Macronutrient breakdown.
   - Highlight days over/under targets.

Challenge 32: Code Snippet Manager

Create a code snippet manager for developers to save and organize code.

Requirements:

1. **Snippet Creation:**
    - Title, description, code content.
    - Select programming language.
    - Add tags for organization.
    - Syntax highlighting in editor.
2. **Organization:**
    - Folders/collections for snippets.
    - Tag-based filtering.
    - Star/favorite snippets.
    - Sort by date, name, language.
3. **Code Display:**
    - Syntax highlighting by language.
    - Line numbers.
    - Copy to clipboard button.
    - Download as file.
4. **Search:**
    - Search by title, description, or code content.
    - Filter by language or tags.
    - Recent snippets section.
5. **Sharing:**
    - Generate shareable link (mock).
    - Public/private toggle.
    - Embed code option.

Challenge 33: Invoice Generator

Build an invoice generator with PDF export functionality.

Requirements:

1. **Invoice Form:**
   - Company details (name, address, logo).
   - Client details (name, email, address).
   - Invoice number (auto-generated).
   - Issue date and due date.
2. **Line Items:**
   - Add multiple items/services.
   - Description, quantity, rate, amount.
   - Auto-calculate line totals.
   - Add/remove line items dynamically.
3. **Calculations:**
   - Subtotal calculation.
   - Tax rate and amount.
   - Discount (percentage or fixed).
   - Grand total.
4. **Preview:**
   - Live invoice preview.
   - Multiple template designs.
   - Print-friendly layout.
5. **Export:**
   - Download as PDF.
   - Send via email (mock).
   - Save draft invoices.

- Invoice history list.

Challenge 34: Fitness Workout Tracker

Build a fitness application to track workouts and exercises.

Requirements:

1. **Exercise Library:**
   - Browse exercises by muscle group.
   - Exercise details with instructions.
   - Animated GIF demonstrations.
   - Add custom exercises.
2. **Workout Builder:**
   - Create workout plans.
   - Add exercises with sets, reps, weight.
   - Rest timer between sets.
   - Reorder exercises.
3. **Workout Session:**
   - Start workout mode.
   - Log actual performance.
   - Auto-start rest timer.
   - Mark exercises complete.
   - Skip exercises option.
4. **Progress Tracking:**
   - Log workout history.
   - Track weight progression per exercise.

- Personal records highlights.
- Weekly/monthly summary charts.

5. **Goals:**
   - Set workout frequency goals.
   - Track goal progress.
   - Streak counter.
   - Achievement badges.

Challenge 35: Podcast Player

Create a podcast player with subscription and episode management.

Requirements:

1. **Podcast Discovery:**
   - Browse podcast categories.
   - Featured and trending podcasts.
   - Search by name or topic.
   - Podcast details with description.
2. **Subscription:**
   - Subscribe/unsubscribe to podcasts.
   - Subscribed podcasts list.
   - New episode indicators.
   - Notification settings per podcast.
3. **Episode List:**
   - Episodes sorted by date.
   - Episode duration and publish date.

- Episode description preview.
- Download episodes (mock).

4. **Player:**
   - Standard audio controls.
   - Playback speed adjustment (0.5x - 2x).
   - Skip forward/back 10/30 seconds.
   - Sleep timer.
   - Remember position on resume.
5. **Queue:**
   - Queue management.
   - Play next functionality.
   - History of played episodes.

Challenge 36: Restaurant Table Reservation

Build a table reservation system for restaurants.

Requirements:

1. **Restaurant Listing:**
   - Browse restaurants with filters.
   - Show cuisine type, rating, price range.
   - Location-based search.
   - Restaurant images and details.
2. **Reservation Form:**
   - Select date and time.
   - Party size selection.

- Show available tables.
- Special requests field.
- Contact information.

3. **Table Visualization:**
   - Visual floor plan (optional).
   - Show table status (available, reserved).
   - Table capacity indicators.
   - Select preferred table.
4. **Confirmation:**
   - Reservation summary.
   - Confirmation email (mock).
   - Calendar integration download.
   - Reservation code.
5. **My Reservations:**
   - Upcoming reservations list.
   - Past reservations history.
   - Modify/cancel reservation.
   - Leave review after visit.

Challenge 37: Newsletter Builder

Create a newsletter/email template builder with drag-and-drop.

Requirements:

1. **Content Blocks:**
   - Header with logo.

- Text block with formatting.
- Image block with caption.
- Button block with link.
- Divider and spacer.
- Social media icons.

2. **Builder Interface:**
   - Drag blocks to canvas.
   - Reorder blocks.
   - Delete blocks.
   - Edit block content inline.
   - Real-time preview.
3. **Styling:**
   - Set colors and fonts.
   - Adjust padding and margins.
   - Background colors per block.
   - Border options.
4. **Templates:**
   - Start from template.
   - Save as template.
   - Template categories.
5. **Export:**
   - Preview in email mode.
   - Export HTML code.
   - Send test email (mock).

Challenge 38: Language Learning Flashcards

Build a language learning app with flashcards and pronunciation.

Requirements:

1. **Vocabulary:**
   - Add words with translation.
   - Include pronunciation (text or audio mock).
   - Example sentences.
   - Categorize by topic.
2. **Learning Modes:**
   - Flashcard flip mode.
   - Multiple choice quiz.
   - Type the translation.
   - Match pairs game.
3. **Progress:**
   - Track mastery level per word.
   - Spaced repetition scheduling.
   - Daily learning goals.
   - Streak tracking.
4. **Lessons:**
   - Organized lesson structure.
   - Progressive difficulty.
   - Lesson completion badges.
   - Review weak areas.
5. **Statistics:**
   - Words learned chart.
   - Time spent learning.

- Accuracy percentage.
- Leaderboard (mock).

Challenge 39: Travel Itinerary Planner

Build a travel planning app with day-by-day itinerary creation.

Requirements:

1. **Trip Creation:**
   - Trip name, destination, dates.
   - Cover image.
   - Trip description.
   - Budget setting.
2. **Itinerary Builder:**
   - Day-by-day planning.
   - Add activities with time slots.
   - Activity categories (food, sightseeing, transport).
   - Add notes and booking references.
   - Map integration (show locations).
3. **Places:**
   - Search and add places.
   - Save place details.
   - Opening hours and contact.
   - Estimated costs.
4. **Budget Tracking:**
   - Expense categories.

- Track spending vs budget.
- Currency conversion.
- Budget breakdown by day/category.

5. **Sharing:**
   - Share trip with others (mock).
   - Collaborative editing (mock).
   - Export as PDF.
   - Packing checklist.

Challenge 40: Product Comparison Tool

Build a product comparison application for e-commerce.

Requirements:

1. **Product Selection:**
   - Search and select products.
   - Add up to 4 products to compare.
   - Quick add from product cards.
   - Recently compared products.
2. **Comparison Table:**
   - Side-by-side feature comparison.
   - Highlight best values.
   - Expandable feature categories.
   - Sticky product headers on scroll.
3. **Features:**
   - Price comparison with history.

- Specification differences.
- Pros and cons lists.
- Rating comparisons.

4. **Save & Share:**
   - Save comparison for later.
   - Share comparison link (mock).
   - Export as PDF/image.
5. **Recommendations:**
   - Show winner summary.
   - Highlight key differences.
   - Similar products suggestions.

Challenge 41: Event Ticketing Platform

Build an event ticketing interface with seat selection.

Requirements:

1. **Event Listing:**
   - Browse upcoming events.
   - Filter by category, date, location.
   - Event cards with image, title, date, venue.
   - Featured events section.
2. **Event Details:**
   - Full event information.
   - Artist/performer details.
   - Venue information with map.

- Ticket pricing tiers.

3. **Ticket Selection:**
   - Visual seat map (for seated events).
   - Section selection.
   - Choose ticket quantity.
   - Show available vs sold seats.
4. **Checkout:**
   - Order summary.
   - Promo code input.
   - Customer details form.
   - Payment integration (mock).
5. **My Tickets:**
   - View purchased tickets.
   - QR code for entry (mock).
   - Download tickets.
   - Transfer tickets (mock).

Challenge 42: Job Application Tracker

Build an application to track job applications and interviews.

Requirements:

1. **Application Entry:**
   - Company name and position.
   - Application date.
   - Status (Applied, Interview, Offer, Rejected).

- Salary range.
- Notes and links.

2. **Pipeline View:**
   - Kanban-style board by status.
   - Drag to update status.
   - Card count per column.
   - Color-coded priorities.
3. **Company Details:**
   - Company information.
   - Contact person details.
   - Interview schedule.
   - Documents attached.
4. **Calendar:**
   - Interview calendar view.
   - Reminders for follow-ups.
   - Deadline tracking.
5. **Analytics:**
   - Application statistics.
   - Success rate by status.
   - Time in each stage.
   - Monthly application trends.

Section 3: Advanced Level

Challenges (8 Challenges)

These challenges require advanced React patterns, Redux/Redux Toolkit, React Router, performance optimization, and complex state management.

Challenge 43: E-Commerce Platform with Redux Toolkit

Build a complete e-commerce platform with Redux Toolkit for state management and React Router for navigation.

Requirements:

1. **Product Management (Redux Slice: `productsSlice`):**
   - Fetch products from API with `createAsyncThunk`.
   - Filter products by category, price range, rating.
   - Sort products by various criteria.
   - Handle loading, success, and error states.
   - Implement product search with debouncing.
2. **Shopping Cart (Redux Slice: `cartSlice`):**
   - Add/remove items with quantity management.
   - Apply coupon codes with validation.
   - Calculate subtotal, tax, shipping, and total.
   - Persist cart to localStorage with Redux middleware.
   - Implement cart synchronization across tabs.
3. **User Authentication (Redux Slice: `authSlice`):**
   - Login/logout functionality.
   - JWT token management.

- Protected routes based on auth state.
- User profile management.
- Address book management.

4. **Checkout Flow (React Router):**
   - Multi-step checkout (`/checkout/cart` , `/checkout/shipping` , `/checkout/payment` , `/checkout/confirm`).
   - Form validation at each step.
   - Order summary sidebar.
   - Order placement with async thunk.
   - Order confirmation page.
5. **Order Management (Redux Slice: `ordersSlice`):**
   - Order history listing.
   - Order detail view.
   - Order tracking status.
   - Reorder functionality.
6. **Routing Structure:**
   - `/` - Home with featured products
   - `/products` - Product listing with filters
   - `/products/:id` - Product detail
   - `/cart` - Shopping cart
   - `/checkout/*` - Checkout flow
   - `/account/*` - User account pages
   - `/orders` - Order history
7. **Performance:**
   - Implement code splitting by route.
   - Use `React.memo` for product cards.
   - Implement virtualized product list for large catalogs.

- Optimize Redux selectors with `createSelector`.

Challenge 44: Movie Database with Redux Toolkit

Build a comprehensive movie database application similar to IMDb.

Requirements:

1. **Movie Catalog (Redux Slice: `moviesSlice`):**
   - Fetch movies with pagination using `createAsyncThunk`.
   - Category-based movie lists (Popular, Top Rated, Upcoming, Now Playing).
   - Advanced filtering and sorting.
   - Infinite scroll implementation.
2. **Search (Redux Slice: `searchSlice`):**
   - Multi-content search (movies, TV shows, people).
   - Search suggestions with debounce.
   - Recent searches history.
   - Search results with type tabs.
3. **User Features (Redux Slice: `userSlice`):**
   - Watchlist management.
   - Favorites list.
   - Rating system.
   - Watch history.
   - Custom lists creation.
4. **Movie Details:**
   - Full movie information.
   - Cast and crew.

- Reviews and ratings.
- Similar movies.
- Trailers and videos.
- Where to watch.

5. **Routing:**

- `/` - Home with carousels
- `/movie/:id` - Movie details
- `/tv/:id` - TV show details
- `/person/:id` - Person details
- `/search` - Search results
- `/genre/:id` - Genre-specific movies
- `/watchlist` - User watchlist
- `/lists` - User custom lists

6. **API Integration:**

- Use TMDB API.
- Implement RTK Query for data fetching.
- Cache management.
- Error handling.

7. **Advanced Features:**

- Dark/Light theme.
- Language selection.
- Region-based content.
- Responsive design.

Challenge 45: Analytics Dashboard with Redux Toolkit

Build a comprehensive analytics dashboard with real-time data visualization and Redux Toolkit.

Requirements:

1. **Dashboard Overview (Redux Slice: `dashboardSlice`):**
   - Fetch dashboard metrics with `createAsyncThunk`.
   - Real-time data polling.
   - Date range filtering.
   - Metric comparison periods.
   - KPI cards with trends.
2. **Charts & Visualization (Redux Slice: `chartsSlice`):**
   - Line charts for trends.
   - Bar charts for comparisons.
   - Pie charts for distributions.
   - Geographic maps for regional data.
   - Real-time updating charts.
3. **Reports (Redux Slice: `reportsSlice`):**
   - Pre-built report templates.
   - Custom report builder.
   - Schedule report generation.
   - Export to PDF/CSV/Excel.
   - Share reports.
4. **Filters (Redux Slice: `filtersSlice`):**
   - Global filter state.
   - Date range picker.
   - Dimension filters.

- Segment selection.
- Filter presets.

5. **Routing:**
   - `/dashboard` - Main overview
   - `/dashboard/traffic` - Traffic analytics
   - `/dashboard/sales` - Sales analytics
   - `/dashboard/users` - User analytics
   - `/reports` - Reports listing
   - `/reports/new` - Report builder
   - `/settings` - Dashboard settings
6. **Performance:**
   - Virtualized data tables.
   - Lazy load charts.
   - Memoized selectors.
   - Optimistic UI updates.
7. **Advanced:**
   - Drill-down functionality.
   - Cross-filtering between widgets.
   - Annotation system.
   - Alert configuration.

Challenge 46: Multi-Tenant SaaS Application

Build a multi-tenant SaaS application with team workspaces, role-based access, and subscription management.

Requirements:

1. **Authentication & Authorization (Redux Slice: `authSlice`):**
    - JWT authentication flow.
    - Role-based access control (Admin, Editor, Viewer).
    - Permission checking throughout app.
    - Session management and token refresh.
    - Multi-factor authentication (mock).
2. **Workspace Management (Redux Slice: `workspaceSlice`):**
    - Create/join multiple workspaces.
    - Workspace switching.
    - Workspace-scoped data isolation.
    - Workspace settings and branding.
    - Workspace deletion with confirmation.
3. **Team Management (Redux Slice: `teamSlice`):**
    - Invite team members.
    - Role assignment and modification.
    - Remove team members.
    - Team activity log.
    - Pending invitations management.
4. **Resource Management (Redux Slice: `resourcesSlice`):**
    - Generic CRUD operations.
    - Resource permissions per role.
    - Audit logging.
    - Soft delete with restore.
    - Resource sharing between workspaces.
5. **Billing (Redux Slice: `billingSlice`):**
    - Subscription plans display.

- Plan upgrade/downgrade.
- Usage tracking against limits.
- Billing history and invoices.
- Payment method management (mock).

6. **Routing:**
   - `/:workspaceId/dashboard` - Workspace dashboard
   - `/:workspaceId/resources` - Resource management
   - `/:workspaceId/team` - Team management
   - `/:workspaceId/settings` - Workspace settings
   - `/account` - User account
   - `/billing` - Billing management
7. **Advanced:**
   - Workspace-scoped routes.
   - Permission-based route guards.
   - Optimistic updates with rollback.
   - RTK Query for API layer.

Challenge 47: Real-Time Collaborative Document Editor

Create a collaborative document editor with real-time synchronization, version history, and commenting.

Requirements:

1. **Editor (Redux Slice: `editorSlice`):**
   - Rich text editing with formatting.
   - Collaborative cursor positions.
   - Auto-save functionality.

- Undo/redo stack.
- Content structure management.

2. **Collaboration (Redux Slice: `collaborationSlice`):**
   - Real-time sync simulation.
   - Presence system (who's viewing).
   - Concurrent edit handling.
   - User join/leave notifications.
   - Edit conflict resolution.
3. **Documents (Redux Slice: `documentsSlice`):**
   - Document CRUD operations.
   - Folder organization.
   - Document sharing with permissions.
   - Recent documents.
   - Document templates.
4. **Version History (Redux Slice: `historySlice`):**
   - Track all changes.
   - Version timeline.
   - Restore previous versions.
   - Version diff comparison.
   - Change attribution.
5. **Comments (Redux Slice: `commentsSlice`):**
   - Inline comments on text.
   - Comment threads and replies.
   - Resolve/unresolve comments.
   - @mentions with notifications.
6. **Routing:**

- `/docs` - Document listing
- `/docs/:docId` - Editor
- `/docs/:docId/history` - Version history
- `/shared` - Shared with me
- `/templates` - Document templates

7. **Export:**
   - Export to PDF, DOCX, Markdown.
   - Import from common formats.
   - Print-friendly view.

Challenge 48: Project Management System

Build a comprehensive project management system like Asana or Monday.com.

Requirements:

1. **Projects (Redux Slice: `projectsSlice`):**
   - Create and manage projects.
   - Project templates.
   - Project settings and permissions.
   - Archive projects.
   - Project status and progress.
2. **Tasks (Redux Slice: `tasksSlice`):**
   - Task CRUD with subtasks.
   - Task assignments.
   - Due dates and priorities.
   - Custom fields.
   - Task dependencies.

- Recurring tasks.

3. **Views (Redux Slice: `viewsSlice`):**
   - List view.
   - Kanban board.
   - Calendar view.
   - Gantt chart view.
   - Timeline view.
   - Custom saved views.
4. **Team (Redux Slice: `teamSlice`):**
   - Team member management.
   - Workload view.
   - Team inbox.
   - @mentions and notifications.
5. **Routing:**
   - `/projects` - Project listing
   - `/projects/:id` - Project detail (multiple view modes)
   - `/projects/:id/tasks/:taskId` - Task detail modal
   - `/my-tasks` - Personal task view
   - `/inbox` - Notifications and updates
   - `/calendar` - Calendar view across projects
6. **Advanced:**
   - Drag-and-drop everywhere.
   - Bulk actions on tasks.
   - Advanced filtering and sorting.
   - Automation rules (mock).
   - Time tracking.

Challenge 49: Full-Stack Clone - GitHub Repository Browser

Build a GitHub-like repository browser with code viewing, issues, pull requests, and repository management.

Requirements:

1. **Repository (Redux Slice: `repoSlice`):**
    - Repository overview.
    - README rendering.
    - Repository statistics.
    - Star/watch/fork functionality.
    - Topics and tags.
2. **Code Browser (Redux Slice: `codeSlice`):**
    - File tree navigation.
    - File content with syntax highlighting.
    - Branch/tag switching.
    - Commit history.
    - Blame view.
    - File history.
3. **Issues (Redux Slice: `issuesSlice`):**
    - Issue listing with filters.
    - Issue creation with markdown.
    - Issue comments.
    - Labels and assignees.
    - Issue state management.
    - Reactions on issues.
4. **Pull Requests (Redux Slice: `pullsSlice`):**

- PR listing and filters.
- PR detail with description.
- File diff view.
- Review comments on lines.
- Merge/close actions.
- CI status (mock).

5. **Commits (Redux Slice: `commitsSlice`):**
   - Commit history with pagination.
   - Commit detail with diff.
   - Browse at specific commit.
   - Commit authorship.
6. **Routing:**
   - `/:owner/:repo` - Repository overview
   - `/:owner/:repo/tree/:branch/*path` - Code browser
   - `/:owner/:repo/issues` - Issues list
   - `/:owner/:repo/issues/:number` - Issue detail
   - `/:owner/:repo/pulls` - Pull requests
   - `/:owner/:repo/pulls/:number` - PR detail
   - `/:owner/:repo/commits/:branch` - Commit history
   - `/:owner/:repo/commit/:sha` - Commit detail
   - `/search` - Global search
7. **API:**
   - Use GitHub REST API.
   - RTK Query for data fetching.
   - Pagination handling.
   - Rate limit management.

Challenge 50: Learning Management System (LMS)

Build a comprehensive learning management system with courses, lessons, quizzes, and progress tracking.

Requirements:

1. **Courses (Redux Slice: `coursesSlice`):**
   - Course catalog with categories.
   - Course detail with curriculum.
   - Course enrollment.
   - Course ratings and reviews.
   - Instructor information.
2. **Lessons (Redux Slice: `lessonsSlice`):**
   - Video lessons with player.
   - Text/article lessons.
   - Downloadable resources.
   - Lesson notes feature.
   - Mark lesson complete.
3. **Quizzes (Redux Slice: `quizzesSlice`):**
   - Multiple question types.
   - Timed quizzes.
   - Instant feedback.
   - Quiz review after submission.
   - Retake options.
4. **Progress (Redux Slice: `progressSlice`):**
   - Course progress tracking.
   - Learning streaks.
   - Certificates on completion.

- Achievement badges.
- Learning statistics.

5. **Social (Redux Slice: `socialSlice`):**
   - Discussion forums per course.
   - Q&A section.
   - Student interactions.
   - Instructor announcements.
6. **Routing:**
   - `/courses` - Course catalog
   - `/courses/:id` - Course detail
   - `/courses/:id/learn` - Learning interface
   - `/courses/:id/lessons/:lessonId` - Lesson view
   - `/courses/:id/quiz/:quizId` - Quiz taking
   - `/my-learning` - Enrolled courses
   - `/certificates` - Earned certificates
   - `/instructor/*` - Instructor dashboard (if applicable)
7. **Features:**
   - Video player with speed control.
   - Picture-in-picture mode.
   - Bookmarks in videos.
   - Note-taking sync with video.
   - Offline access (mock).

Tips for Completing These Challenges

1. **Start with Planning:** Before coding, plan your component structure, state management, and routing architecture.
2. **Build Incrementally:** Implement one feature at a time. Get the basics working before adding advanced features.
3. **Use TypeScript:** For advanced challenges, consider using TypeScript for better type safety and developer experience.
4. **Focus on User Experience:** Handle loading states, errors, and edge cases gracefully.
5. **Write Clean Code:** Use consistent naming, organize files logically, and add comments where needed.
6. **Test Your Work:** Write unit tests for reducers and integration tests for user flows.
7. **Make It Responsive:** Ensure your application works well on different screen sizes.
8. **Optimize Performance:** Use React DevTools and Redux DevTools to identify and fix performance issues.
9. **Document Your Code:** Write clear README files explaining how to run and use your application.
10. **Version Control:** Use Git with meaningful commit messages to track your progress.

Challenge Difficulty Summary

Level	Count	Focus Areas
Beginner	20	Components, Props, State, Basic Hooks, localStorage
Mid-Level	22	Context API, Custom Hooks, API Integration, Performance
Advanced	8	Redux Toolkit, React Router, Complex State, Full Applications

---

Good luck with your React interview preparation! 🚀

Created by Yogesh Chavan

yogeshchavan.dev | LinkedIn | My All Courses, Ebooks, Webinars